```
function goodMorning()
{
    print "Good morning!" . PHP_EOL;
}
```

```
Output:
Good morning!
PHP Fatal error: Uncaught Error: Call to undefined function helloWorld()
```

Function Arguments and Returning Data

We've already examined the distinction between functions and sub-routines and the fact that PHP does not distinguish between them syntactically. The examples in the previous section were technically sub-routines; they did not accept input or return output. By adding a little more information to their declarations PHP functions can also accept data as input for use as named variables inside the function code block. Values are assigned when the function is called, and the calling code (usually) passes in the data.

Arguments and Parameters

To enable the passing of data into our functions we first add one or more variable names between the parentheses when the function is declared, separated by commas. Those variables can then be used inside the braces that define the function code block and the calling code will supply the data. The variables named in a function declaration in this way are called *parameters*.

Code that calls the function can then put expressions into the parentheses, the values of which are made available in the function body using the parameter names. The expressions inside the parentheses of the calling code are called *arguments*. It's quite common to see the words parameter and argument used interchangeably when discussing functions.

```
/*
 * $bar is a parameter, code inside the braces can use the
 * $bar variable to access the data sent by the caller
 */
```